

● SESIÓN 9 Filtros combinados PRO con Specification

"Si hay muchos filtros, no uses if/else: usa Specifications."

■ Bloque 1 — Problema real

En sesión 8 hicimos:

- author o title o category

Pero en APIs reales necesitamos:

- category=programming
- author=martin
- title=clean
- todos juntos, opcionales
- paginación y ordenación

 **✓ Objetivo:** construir un único endpoint que combine filtros dinámicos de forma limpia.

■ Bloque 2 — Teoría esencial

¿Qué es una Specification?

Una Specification es un "filtro" que se convierte en un Predicate SQL (WHERE).

Ventajas:

- **Componible:** and(), or()
- **Escalable:** añades filtros sin reescribir todo
- **Limpio y mantenible**

Requisito

Tu repo debe extender:

JpaSpecificationExecutor

■ Bloque 3 — Estructura del proyecto

```
com.cladsb1.books
├── controller
│   └── BookController
├── service
│   └── BookService
├── repository
│   ├── BookRepository  ✓ extends JpaSpecificationExecutor
│   └── CategoryRepository
├── entity
│   ├── Book
│   └── Category
├── dto
│   ├── request
│   │   └── BookFilterDTO  ✓ NUEVO (filtros)
│   └── response
│       ├── BookResponseDTO
│       └── PageResponseDTO
├── spec
│   └── BookSpecifications  ✓ NUEVO
└── error
```

■ Bloque 4 — Repository: habilitar Specifications

📁 repository/BookRepository.java

```
package com.cladsb1.books.repository;

import com.cladsb1.books.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface BookRepository
    extends JpaRepository<Book, Long>,
        JpaSpecificationExecutor<Book> {
}
```



Bloque 5 — DTO de filtros



dto/request/BookFilterDTO.java

```
package com.cladsb1.books.dto.request;
```

```
/**
 * DTO de filtros (query params).
 * Todo es opcional.
 */
```

```
public record BookFilterDTO(
    String title,
    String author,
    String category
) {}
```

✓ A) VERSIÓN CON LAMBDAS/STREAMS



Bloque 6A — Specifications (CON lambdas)



spec/BookSpecifications.java

```
package com.cladsb1.books.spec;
```

```
import com.cladsb1.books.entity.Book;
import org.springframework.data.jpa.domain.Specification;
```

```
public class BookSpecifications {
```

```
    private BookSpecifications() {}
```

```
    public static Specification<Book> titleContainsIgnoreCase(String title) {
        return (root, query, cb) ->
            cb.like(cb.lower(root.get("title")),
                "%" + title.toLowerCase() + "%");
    }
```

```
    public static Specification<Book> authorEqualsIgnoreCase(String author) {
        return (root, query, cb) ->
            cb.equal(cb.lower(root.get("author")),
                author.toLowerCase());
    }
```

```
    public static Specification<Book> categoryNameEqualsIgnoreCase(String category) {
        // Navegación por relación: book.category.name
        return (root, query, cb) ->
            cb.equal(cb.lower(root.get("category").get("name")),
                category.toLowerCase());
    }
}
```



Bloque 7A — Service: construir spec combinada (CON lambdas)



service/BookService.java (añade este método)

```
package com.cladsb1.books.service;

import com.cladsb1.books.dto.request.BookFilterDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.dto.response.PageResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.spec.BookSpecifications;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;

@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public PageResponseDTO<BookResponseDTO> search(BookFilterDTO filter, Pageable pageable) {
        Specification<Book> spec = Specification.where(null);

        if (filter.title() != null && !filter.title().isBlank()) {
            spec = spec.and(BookSpecifications.titleContainsIgnoreCase(filter.title()));
        }
        if (filter.author() != null && !filter.author().isBlank()) {
            spec = spec.and(BookSpecifications.authorEqualsIgnoreCase(filter.author()));
        }
        if (filter.category() != null && !filter.category().isBlank()) {
            spec = spec.and(BookSpecifications.categoryNameEqualsIgnoreCase(filter.category()));
        }

        Page<Book> page = bookRepository.findAll(spec, pageable);

        return new PageResponseDTO<>(
            page.getContent().stream().map(this::toDTO).toList(),
            page.getNumber(),
            page.getSize(),
            page.getTotalElements(),
            page.getTotalPages(),
            page.isLast()
        );
    }

    private BookResponseDTO toDTO(Book b) {
        return new BookResponseDTO(
            b.getId(),
            b.getTitle(),
            b.getAuthor(),
            b.getCategory().getName()
        );
    }
}
```



Bloque 8A — Controller: endpoint combinado



controller/BookController.java

```
@GetMapping("/search")
public ResponseEntity<PageResponseDTO<BookResponseDTO>> search(
    @RequestParam(required = false) String title,
    @RequestParam(required = false) String author,
    @RequestParam(required = false) String category,
    Pageable pageable
) {
    return ResponseEntity.ok(
        bookService.search(new BookFilterDTO(title, author, category), pageable)
    );
}
```

Uso en Swagger:

/api/books/search?title=clean&category=programming&page=0&size=5&sort=title,asc

✓ B) VERSIÓN SIN LAMBDA/STREAMS



Bloque 6B — Specifications SIN lambdas (clases anónimas)

*Nota: Specification en sí usa lambdas porque es una interfaz funcional, pero podemos escribirlo con **clases anónimas** (más "clásico").*



spec/BookSpecifications.java (SIN lambdas)

```
package com.cladsb1.books.spec;

import com.cladsb1.books.entity.Book;
import org.springframework.data.jpa.domain.Specification;
import jakarta.persistence.criteria.*;

public class BookSpecifications {

    private BookSpecifications() {}

    public static Specification<Book> titleContainsIgnoreCase(final String title) {
        return new Specification<Book>() {
            @Override
            public Predicate toPredicate(Root<Book> root, CriteriaQuery<?> query, CriteriaBuilder cb) {
                return cb.like(cb.lower(root.get("title")),
                    "%" + title.toLowerCase() + "%");
            }
        };
    }

    public static Specification<Book> authorEqualsIgnoreCase(final String author) {
        return new Specification<Book>() {
            @Override
            public Predicate toPredicate(Root<Book> root, CriteriaQuery<?> query, CriteriaBuilder cb) {
                return cb.equal(cb.lower(root.get("author")),
                    author.toLowerCase());
            }
        };
    }

    public static Specification<Book> categoryNameEqualsIgnoreCase(final String category) {
        return new Specification<Book>() {
            @Override
            public Predicate toPredicate(Root<Book> root, CriteriaQuery<?> query, CriteriaBuilder cb) {
                return cb.equal(cb.lower(root.get("category").get("name")),
                    category.toLowerCase());
            }
        };
    }
}
```



Bloque 7B — Service SIN streams

```
public PageResponseDTO<BookResponseDTO> search(BookFilterDTO filter, Pageable pageable) {
    Specification<Book> spec = Specification.where(null);

    if (filter.title() != null && !filter.title().isBlank()) {
        spec = spec.and(BookSpecifications.titleContainsIgnoreCase(filter.title()));
    }
    if (filter.author() != null && !filter.author().isBlank()) {
        spec = spec.and(BookSpecifications.authorEqualsIgnoreCase(filter.author()));
    }
    if (filter.category() != null && !filter.category().isBlank()) {
        spec = spec.and(BookSpecifications.categoryNameEqualsIgnoreCase(filter.category()));
    }

    Page<Book> page = bookRepository.findAll(spec, pageable);

    // Mapping SIN streams
    List<BookResponseDTO> content = new ArrayList<>();
    for (Book b : page.getContent()) {
        content.add(toDTO(b));
    }

    return new PageResponseDTO<>(
        content,
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages(),
        page.isLast()
    );
}
```



Bloque 9 — Tabla de ejemplos

Request	Qué hace
/api/books/search?page=0&size=5	devuelve todo paginado
/api/books/search?category=programming	filtra por categoría
/api/books/search?title=clean	contains en título
/api/books/search? author=martin&category=programming	combinación AND
/api/books/search?title=java&sort=title,asc	filtro + sort



Bloque 10 — Errores típicos



"if/else" infinito para filtros combinados

No uses cadenas interminables de condicionales para manejar múltiples filtros



Exponer entities

Se mantiene DTO - nunca expongas las entidades directamente



Meter lógica de filtros en controller

Debe ir al service/spec - mantén el controller limpio



Olvidar JpaSpecificationExecutor

Sin esta interfaz, las Specifications no funcionarán



Bloque 11 — Cierre

"Specifications permiten filtros combinados sin convertir tu código en un infierno de ifs."



Mensaje clave: Con Specifications, tus filtros son componibles, escalables y mantenibles.